Requirement ID	Requirement Wording	FAA Requirement ID	DOT/FAA/AR-06/54 Appenix B Requirement Text	Applicable to C?	Unique C Lannguage Test Case Required?
		(C.1)	Verify that statement coverage of basic blocks is indicated correctly.	Yes	No - Handled by Requirements within this subsection
		(C.1.1)	Verify that basic blocks are correctly detected.	Yes	No - Handled by Requirements within this subsection
C-1	Verify that basic blocks in subprograms are correctly detected.	(C.1.1.1)	Verify that basic blocks in subprograms are correctly detected.	Yes	Yes
C-2	Verify that basic blocks forming a handled-sequence-of-statements in a function body are correctly detected.	(C.1.1.2)	Verify that basic blocks forming a handled-sequence-of-statements in a package body are correctly detected.	Yes	Yes
	Verify that compound statements are correctly handled.	(C.1.2)	Verify that compound statements are correctly handled.	Yes	No - Handled by Requirements within this subsection
C-3	Verify that loops are correctly handled.	(C.1.2.1)	Verify that loops are correctly handled.	Yes	Yes
	Verify that conditional statements are correctly handled.	(C.1.2.2)	Verify that conditional statements are correctly handled.	Yes	No - Handled by Requirements within this subsection
C-4	Verify that if statements are correctly handled, including variants such as if-else and if-elsif-else.	(C.1.2.2.1)	Verify that if statements are correctly handled, including variants such as if-else and if-elsif-else.	Yes	Yes
C-5	Verify that case statements are correctly handled.	(C.1.2.2.2)	Verify that case statements are correctly handled.	Yes	Yes
	Verify that coverage is correctly reported in the presence of exception mechanisms in Ada.	(C.1.3)	Verify that coverage is correctly reported in the presence of exception mechanisms in Ada.	No	No - Ada Only
	Verify that coverage is correctly reported in the presence of exception handlers in Ada.	(C.1.3.1)	Verify that coverage is correctly reported in the presence of exception handlers in Ada.	No	No - Ada Only
	Verify that (apparent) basic blocks that do not complete execution because of exceptions or “long jumps” are handled correctly.	(C.1.3.2)	Verify that (apparent) basic blocks that do not complete execution because of exceptions or “long jumps” are handled correctly.	No	No - Ada Only
	Verify that (apparent) basic blocks that do not complete execution to the end are handled correctly.	(C.1.4)	Verify that (apparent) basic blocks that do not complete execution to the end are handled correctly.	Yes	No - Handled by Requirements within this subsection
C-6	Verify that (apparent) basic blocks that do not complete execution because of a break statement are handled correctly.	(C.1.4.1)	Verify that (apparent) basic blocks that do not complete execution because of exit or break statements are handled correctly.	Yes	Yes
C-7	Verify that (apparent) basic blocks that do not complete execution because of goto statements are handled correctly.	(C.1.4.2)	Verify that (apparent) basic blocks that do not complete execution because of goto statements are handled correctly.	Yes	Yes
C-8	Verify that (apparent) basic blocks that do not complete execution because of return statements are handled correctly.	(C.1.4.3)	Verify that (apparent) basic blocks that do not complete execution because of return statements are handled correctly.	Yes	Yes
C-9	Verify that declarations that generate executable code are included in statement coverage report.	(C.1.5)	Verify that declarations that generate executable code are included in statement coverage report.	Yes	Yes
	Verify that partial evaluation does not impact statement coverage.	(C.2)	Verify that partial evaluation does not impact statement coverage.	Yes	No - Handled by Requirements within this subsection
C-10	Verify that partial evaluation of short-circuited expressions does not impact statement coverage.	(C.2.1)	Verify that partial evaluation of short-circuited expressions does not impact statement coverage.	Yes	Yes

C-11	Verify that partial evaluation of conditional expressions does not impact statement coverage.	(C.2.2)	Verify that partial evaluation of conditional expressions does not impact statement coverage.	Yes	Yes - C only, not Ada
	Verify that coverage reported is consistent with the semantics of the code.	(C.3)	Verify that coverage reported is consistent with the semantics of the code.	Yes	No - Handled by Requirements within this subsection
	Verify that various textual forms of basic blocks are handled correctly.	(C.3.1)	Verify that various textual forms of basic blocks are handled correctly.	Yes	No - Handled by Requirements within this subsection
C-12	Verify that a basic block spanning multiple lines is handled correctly.	(C.3.1.1)	Verify that a basic block spanning multiple lines is handled correctly.	Yes	Yes
C-13	Verify that a basic block spanning a single line is handled correctly.	(C.3.1.2)	Verify that a basic block spanning a single line is handled correctly.	Yes	Yes
C-14	Verify that multiple basic blocks on one line are handled correctly.	(C.3.1.3)	Verify that multiple basic blocks on one line are handled correctly.	Yes	Yes
C-15	Verify that the coverage reported is not affected by source code indentation.	(C.3.1.4)	Verify that the coverage reported is not affected by source code indentation.	Yes	Yes
	Verify that keywords are not used out of context in reporting coverage.	(C.3.1.5)	Verify that keywords are not used out of context in reporting coverage.	No	No - Ada Only
	Verify that overloading of names does not impact coverage reported.	(C.3.2)	Verify that overloading of names does not impact coverage reported.	No	No - Ada Only
	Verify that separately compiled bodies do not impact coverage.	(C.3.3)	Verify that separately compiled bodies do not impact coverage.	No	No - Ada Only
	Verify that nesting of program units within each other is handled correctly.	(C.3.4)	Verify that nesting of program units within each other is handled correctly.	No	No - Ada Only
	[Ada Only] Verify that generics are handled correctly.	(C.4)	[Ada Only] Verify that generics are handled correctly.	No	No - Ada Only
	Verify that statement coverage is reported correctly for a single instantiation.	(C.4.1)	Verify that statement coverage is reported correctly for a single instantiation.	No	No - Ada Only
	Verify that statement coverage is reported correctly for multiple instantiations.	(C.4.2)	Verify that statement coverage is reported correctly for multiple instantiations.	No	No - Ada Only
	Verify that compiler optimizations do not impact coverage reported.	(C.5)	Verify that compiler optimizations do not impact coverage reported.	Yes - modified	No - Handled by Requirements within this subsection
C-16	Verify killed assignment elimination is properly reflected with and without compiler optimization.	(C.5.1)	Verify killed assignment elimination does not impact coverage.	Yes - modified	Yes - Compiler optimization requirements were modified to reflect expected gcc behavior in presence of optimization and no optimization. In gcc, optimization analysis should be done with separate tool and no optimization (-O0) should be used for coverage analysis by design.
C-17	Verify that common subexpression optimization is properly reflected with and without compiler optimization.	(C.5.2)	Verify that common subexpression optimization does not impact coverage.	Yes - modified	Yes - same optimization comment
C-18	Verify that loop invariant optimization is reflected properly with and without compiler optimization.	(C.5.3)	Verify that loop invariant optimization does not impact coverage.	Yes - modified	Yes - same optimization comment
C-19	Verify that dead code elimination optimization does not impact coverage.	(C.5.4)	Verify that dead code elimination optimization does not impact coverage.	Yes - modified	Yes - same optimization comment
C-20	Verify that macro and function inlining is properly reflected in coverage.	(C.5.5)	Verify that subprogram inlining optimization does not impact coverage.	Yes	Yes

C-21	Verify that the continue statement is handled correctly.	N/A		Yes	Yes
C-22	Verify that the comma operator ',' including conditionals inside of comma	N/A		Yes	Yes
C-23	Verify that conditional compilation (#ifdef) is properly reflected in coverage.	N/A		Yes	Yes
	Verify that the statement coverage report is correct in the presence of exceptions.	(C.5.6)	Verify that the statement coverage report is correct in the presence of exceptions.	No	No - Handled by Requirements within this subsection
	Verify that coverage reported is consistent with the handling of exceptions.	(C.5.7)	Verify that coverage reported is consistent with the handling of exceptions.	Yes	No - Handled by Requirements in previous sections. The C programming language does not have exception handling features like Ada, or other languages. Conditions which would cause program exceptions, as specified in programming guidelines, should be checked within the program itself with proper return values reported reflecting the error. Because this checking is within normal "if-then-else" programming, no unique test is needed to test for C in-program exception handling. Checks for divide by zero and null pointers should be done within the code. Processor exceptions raised during the execution of a C program that cause a "crash", result in no coverage report. Coverage analysis is only reported upon completed execution of the program being analyzed. This behavior is by design of the gcc coverage.
		(B.1)	Verify that Boolean expressions in branching constructs are handled correctly.	Yes	No - Handled by Requirements within this subsection
		(B.1.1)	Verify that simple Boolean expressions are handled correctly.	Yes	No - Handled by Requirements within this subsection
B-1	Verify that Boolean variable references (local variable, global variable, parameter, array object, struct object) are handled correctly.	(B.1.1.1)	Verify that Boolean variable references are handled correctly.	Yes	Yes
B-2	Verify that calls to functions that return Boolean results are handled correctly.	(B.1.1.2)	Verify that calls to functions that return Boolean results are handled correctly.	Yes	Yes
	Verify that relational and range test expressions are handled correctly.	(B.1.1.3)	Verify that relational and range test expressions are handled correctly.	No	No - Ada Only
	Verify that Boolean attribute references are handled correctly.	(B.1.1.4)	Verify that Boolean attribute references are handled correctly.	No	No - Ada Only
B-3	Verify that constant Boolean values are handled correctly.	(B.1.1.5)	Verify that constant Boolean values are handled correctly.	Yes	Yes
	Verify that complex Boolean expressions are handled correctly.	(B.1.2)	Verify that complex Boolean expressions are handled correctly.	Yes	No - Handled by Requirements within this subsection
B-4	Verify that Boolean expressions containing the unary not operator are handled correctly.	(B.1.2.1)	Verify that Boolean expressions containing the unary not operator are handled correctly.	Yes	Yes
B-5	Verify that Boolean expressions containing the binary operators and and or are handled correctly.	(B.1.2.2)	Verify that Boolean expressions containing the binary operators and and or are handled correctly.	Yes	Yes

B-6	Verify that Boolean expressions containing relational operators (==, !=, >, <, >=, <=) are handled correctly.	(B.1.2.3)	Verify that Boolean expressions containing relational operators are handled correctly.	Yes	Yes
B-7	Verify that Boolean expressions containing the C ternary Boolean operator (A ? B : C) are handled correctly.	(B.1.2.4)	Verify that Boolean expressions containing higher-order operators are handled correctly.	Yes	Yes
		(B.1.2.5)	Verify that Boolean expressions containing multiple operators are handled correctly.	Yes	No - Handled by Requirements within this subsection
		(B.2)	Verify that Boolean expressions in nonbranching constructs are handled correctly.	Yes	No - Handled by Requirements within this subsection
B-8	Verify that Boolean expressions in assignment statements are handled correctly.	(B.2.1)	Verify that Boolean expressions in assignment statements are handled correctly.	Yes	Yes
B-9	Verify that Boolean for loop index variables are handled correctly.	(B.2.2)	Verify that Boolean for loop index variables are handled correctly.	Yes	Yes
B-10	Verify that Boolean actual parameters are handled correctly.	(B.2.3)	Verify that Boolean actual parameters are handled correctly.	Yes	Yes
		(B.2.4)	[Ada only] Verify that Boolean expressions used as “default expressions” in subprogram parameter specifications, discriminant specifications, and record component specifications are handled correctly.	No	No - Ada Only
		(B.2.4.1)	Verify that Boolean expressions used in “default expressions” are monitored for coverage at the point where the expression is written.	No	No - Ada Only
		(B.2.4.2)	Verify that Boolean expressions used in “default expressions” are monitored for coverage at each point where the expression is implicitly evaluated.	No	No - Ada Only
		(B.2.5)	[Ada only] Verify that Boolean expressions used in various other nonbranching contexts are handled correctly.	No	No - Ada Only
		(B.3)	[Ada only] Verify that derived Boolean types are handled correctly.	No	No - Ada Only
		(B.3.1)	Verify that derived Boolean types are handled correctly in branching contexts.	No	No - Ada Only
		(B.3.2)	Verify that derived Boolean types are handled correctly in nonbranching contexts.	No	No - Ada Only
		(B.4)	[Ada only] Verify that Boolean operators are handled correctly under overloading and renaming.	No	No - Ada Only
		(B.4.1)	Verify that overloaded Boolean operators are handled correctly in branching contexts.	No	No - Ada Only
		(B.4.2)	Verify that overloaded Boolean operators are handled correctly in nonbranching contexts.	No	No - Ada Only
		(B.5)	Verify that entry and exit points are handled correctly.	Yes	No - Handled by Requirements within this subsection
		(B.5.1)	Verify that subprogram entry points are handled correctly.	Yes	No - Met by tests in Level C
		(B.5.1.1)	Normal (top of subprogram) entry points.	Yes	No - Met by tests in Level C

		(B.5.1.2)	Implicit entry points (exception handlers).	Yes	No - Met by tests in Level C
		(B.5.2)	Verify that subprogram exit points are handled correctly.	Yes	No - Handled by Requirements within this subsection
		(B.5.2.1)	Explicit return statements.	Yes	No - Met by tests in Level C
B-11	Verify that an implicit return statement is handled correctly in coverage results.	(B.5.2.2)	Verify that subprogram exit points are handled correctly - Implicit return statements.	Yes	Yes
		(B.5.2.3)	RAISE statements.	No	No - Ada Only
		(B.5.2.4)	Implicit exit points (via exceptions raised implicitly).	No	No - Ada Only
		(B.5.3)	Verify that construct (low level) entry points are handled correctly.	Yes	No - per FAA/AR-06/54: "statement coverage implies coverage of the construct entry points."
		(B.5.4)	Verify that construct (low level) exit points are handled correctly.	Yes	No - Per FAA/AR-06/54: "the coverage of construct exit points in Ada or C can be deduced from statement coverage and Boolean expression coverage (each decision takes on all possible outcomes), and therefore do not warrant special consideration."
		(A.1)	Boolean expressions in branching contexts.	Yes	No - Handled by Requirements within this subsection
		(A.1.1)	Boolean expressions in branching contexts - Simple Boolean expressions.	Yes	No - This reduces to simple decision coverage, covered elsewhere
		(A.1.2)	Boolean expressions in branching contexts - Boolean expressions containing Boolean operators.	Yes	No - Handled by Requirements within this subsection
		(A.1.2.1)	Boolean expressions in branching contexts. - Boolean expressions containing uncoupled terms only.The tests in this category verify that Boolean expressions containing Boolean terms, each of which can be set independently of the others, are handled correctly. This category represents the cases in which the five variants of MCDC, namely, UCM, BBM, CCM, MSM, and USM, converge in their interpretation of the DO-178B wording. Consequently, the tests in this category represent the least common denominator that all Level A structural coverage analysis tools should be expected pass.	Yes	No - Handled by Requirements within this subsection
A-1	Verify that uncoupled Boolean expressions containing operators and (&&), or (||), and not (!), each of which can be set independently of the others, are handled correctly.	(A.1.2.1.1)	Boolean expressions in branching contexts. - Boolean expressions containing operators and, or, and not.	Yes	Yes
A-2	Verify that uncoupled Boolean expressions containing other binary operators, each of which can be set independently of the others, are handled correctly. Other binary operators are the following: == (equal to), != (not equal to), > (greater than), < (less than), >= (greater than or equal to), and <= (less than or equal to).	(A.1.2.1.2)	Boolean expressions in branching contexts.Boolean expressions containing other binary operators.	Yes	Yes

A-3	Verify that uncoupled Boolean expressions containing short-circuit operators, each of which can be set independently of the others, are handled correctly.	(A.1.2.1.3)	Boolean expressions in branching contexts.Boolean expressions containing short-circuit operators.	Yes	Yes
A-4	Verify that coupled Boolean expressions containing operators and (&&), or (||), and not (!), each of which can be set independently of the others, are handled correctly.	(A.1.2.2.1)	Boolean expressions in branching contexts.Boolean expressions containing operators and, or, and not.	Yes	Yes
		(A.1.2.2)	Boolean expressions in branching contexts. Boolean expressions containing coupled terms.	Yes	No - Handled by Requirements within this subsection
A-5	Verify that coupled Boolean expressions containing other binary operators, each of which can be set independently of the others, are handled correctly. Other binary operators are the following: == (equal to), != (not equal to), > (greater than), < (less than), >= (greater than or equal to), and <= (less than or equal to).	(A.1.2.2.2)	Boolean expressions in branching contexts.Boolean expressions containing other binary operators.	Yes	Yes
A-6	Verify that coupled Boolean expressions containing short-circuit operators, each of which can be set independently of the others, are handled correctly.	(A.1.2.2.3)	Boolean expressions in branching contexts.Boolean expressions containing short-circuit operators.	Yes	Yes
	Boolean expressions in nonbranching contexts.		(A.2)	Yes	No - Per FAA/AR-06/54, "an elaborate set of MCDC tests for the nonbranching contexts is not necessary because the Level B tests already include tests for coverage of decisions in nonbranching contexts."
A-7	Verify that expressions with 64 conditions function properly, and a compiler warning is issued above 64.	N/A	N/A	Yes	Yes - Additional test for known design features of gcc implementation.
A-8	Verify that non-tree-like decision tree structures are handled correctly.	N/A		Yes	Yes - Additional test for known additions in GCC v14 and beyond. Incoporating 'tree like' into the bdd was the major feature changed to enable MC/DC condition tracking. This should have dedicated tests.
A-9	Verify proper handling of a random assortment of multi-line conditions with multiple variables, parenthesis, and mixed operators.	N/A		Yes	Yes. There are several stress cases and real-world examples in the GCC testsuite that should be carried through, and are mapped to this requirement.